

SE3082

AWS EC2 Lab - Screenshots and Calculations

Lab 06

Exercise 5: Parallelizeing a serial code for fibonacci series using task parallelization

IT24610328

On instance with 2 vCPUs (t3.micro)

Nano - creating the serial fibonacci series code

```
GNU nano 6.2      ex5_fib_serial.c *
#include <stdio.h>
#include <omp.h>

int fib(int n) {
    int i, j;
    if (n<2)
        return n;
    else {
        i=fib(n-1);
        j=fib(n-2);
        return i+j;
    }
}

int main() {
    int n = 30;
    double tstart, tstop, tcalc;

    tstart = omp_get_wtime();
    int result = fib(n);
    tstop = omp_get_wtime();
    tcalc = tstop - tstart;

    printf("Fibonacci(%d) = %d\n", n, result);
    printf("Time = %f seconds\n", tcalc);

    return 0;
}
```

Nano - parallelizing the previously serial fibonacci series code using task parallelization

```
GNU nano 6.2      ex5_fib_task.c
#include <stdio.h>
#include <omp.h>

int fib(int n) {
    int i, j;
    if (n<2)
        return n;
    else {
        #pragma omp task shared(i)
        i=fib(n-1);
        #pragma omp task shared(j)
        j=fib(n-2);
        #pragma omp taskwait
        return i+j;
    }
}

int main() {
    int n = 30;
    int result;
    double tstart, tstop, tcalc;

    tstart = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            result = fib(n);
        }
    }

    tstop = omp_get_wtime();
    tcalc = tstop - tstart;

    printf("Fibonacci(%d) = %d\n", n, result);
    printf("Threads used: %d\n", omp_get_max_threads());
    printf("Time = %f seconds\n", tcalc);

    return 0;
}
```

Output from running the serial Fibonacci series code

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex5_fib_serial.o  
Fibonacci(30) = 832040  
Time = 0.007252 seconds
```

Output from running the parallelized Fibonacci series code

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex5_fib_task.o  
Fibonacci(30) = 832040  
Threads used: 2  
Time = 1.028323 seconds
```

Note : Here the parallelized code is notably slower than the serial version as each Fibonacci call generates a big number of small cheap functions and wrapping each one in pragma omp task introduces massive overhead which vastly exceeds any meaningful speedup the parallelization itself offers.